

# COMP0222/COMP0249 Lab 09: 2D Occupancy Grid Mapping

---

10th March, 2026 ver. 20260310

## 1 Scope and Purpose

In the previous lab, you gained hands-on experience with a 2D scanning laser rangefinder, the RPLIDAR A2M12. In this lab, you will move beyond simple point cloud visualization to implement a Probabilistic Occupancy Grid (OGM) framework. We will distinguish between free space and unknown space, using ray-casting to explicitly model free space, obstacles, and unknown areas. By developing a Scan-to-Map matching algorithm (a simplified Hector SLAM approach, with a Hill-Climbing optimization approach), you will enable the sensor to estimate its own motion purely from laser data, without the need for wheel encoders or IMUs. You will integrate these components into a real-time odometry and mapping system. You will explore practical challenges such as map drift and dynamic object filtering.

## 2 Scenario

Motion in a 2D plane is characterized by a rigid body transformation consisting of translation  $(x, y)$  and rotation  $(\theta)$ . This forms the robot's pose vector:  $x = [x, y, \theta]^T$ . In this lab, you will perform Laser Odometry and building an Occupancy Grid Map. Unlike wheel odometry, which counts wheel revolutions and slips on loose terrain, laser odometry estimates motion by comparing what the robot sees now to what it saw a moment ago. By aligning the current laser scan with a cumulative point cloud (the map), you can estimate the sensor's change in pose. This technique is fundamental for autonomous service robots, such as vacuum cleaners and warehouse automated guided vehicles (AGVs).

### 3 Sensor Specification: RPLIDAR A2M12

The RPLIDAR A2 is a 360-degree Laser Range Scanner (LIDAR), see Fig. 1. It operates based on the principle of Laser Triangulation.

- **Mechanism:** The system emits a modulated infrared laser signal. The laser reflection is captured by a vision acquisition system. By analyzing the position of the focused spot on the image sensor, the distance to the object is calculated.
- **Rotation:** The rangefinder core rotates clockwise to perform a 360-degree omnidirectional scan.
- **Key Specifications:**
  - Distance Range: 0.2m – 12m
  - Angular Resolution:  $\approx 0.25^\circ$
  - Sample Rate: Up to 8000 - 16000 samples/second
  - Scan Rate: 5-15Hz, Typical 10Hz
  - Connection: USB (UART to USB converter included)



Figure 1: RPLIDAR A2M12 and its components.

### 4 Requirements

This lab requires Python 3.8+. You must install the following libraries. It is recommended to pin NumPy to version 1.26.x if errors occur.

```
pip install rplidar-robotics pygame
# Optional compatibility fix if using older SciPy versions:
# pip install "numpy<2", or to be specific pip install numpy==1.26.4
```

These are the same packages that were used in the previous lab.

## 5 Activities

### Activity 0: Basic Connectivity and Visualization

This activity is similar to the previous lab. If you have already tested your sensor, you may skip this step, and go to the next activity.

- **Objective:** Establish communication with the sensor and visualize the raw polar coordinate data  $(r, \theta)$  converted into Cartesian space  $(x, y)$ .
- **Instructions:**
  - Connect the RPLIDAR to your computer via USB.
  - Identify the COM port (Windows) or `/dev/ttyUSB*` path (Linux/Mac). If needed, you will need to change the port number.
  - Run the following Python script using the pygame library. This script handles the coordinate transformation from Polar to Cartesian and renders the points in real-time.
- **Code:** `rplidar_viewer.py`. (provided in the previous lab)

Once the code runs successfully, you will see the scan as shown in Fig. 2.

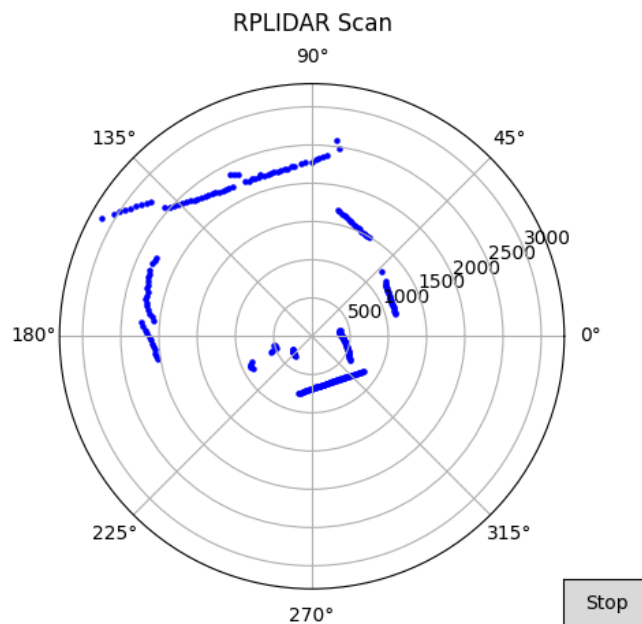


Figure 2: RPLIDAR A2M12 scan viewer.

## Activity 1: Building Occupancy Grid Map from a Single Scan

Concept: The "Gray World" Hypothesis

A raw Lidar scan provides a list of obstacle points. However, for navigation, knowing where obstacles are not (Free Space) is just as important as knowing where they are.

We model the world as a grid of cells  $C_{x,y}$ . Each cell holds a probability value  $P(occupied)$ :

- Gray (0.5): Unknown. The robot has never seen this area.
- White ( $\rightarrow 0.0$ ): Free. Laser rays have passed through here multiple times without hitting anything.
- Black ( $\rightarrow 1.0$ ): Occupied. The laser consistently returns a reflection from this location.

### The Simplified "Visual Accumulation" Model

While formal probabilistic mapping often uses Log-Odds updates, for this high-speed Python implementation we use a simplified linear model:

#### 1. Inverse Sensor Model:

- If a ray passes through a pixel:  $Value_{new} = Value_{old} - \delta_{free}$  (Brighten).
- If a ray hits a pixel:  $Value_{new} = Value_{old} + \delta_{occ}$  (Darken).

#### 2. Saturation: Values are clamped between 0 (Free) and 1 (Occupied).

### Instructions

#### 1. Setup: Connect the Lidar and run the script `occupancy_grid_map_still_lidar.py`.

#### 2. Static Mapping: Keep the sensor perfectly still on the table.

- Observation: Watch how the map initializes as a gray fog. As the Lidar spins, the "rays" carve out white free space. The walls should resolve into sharp black lines.

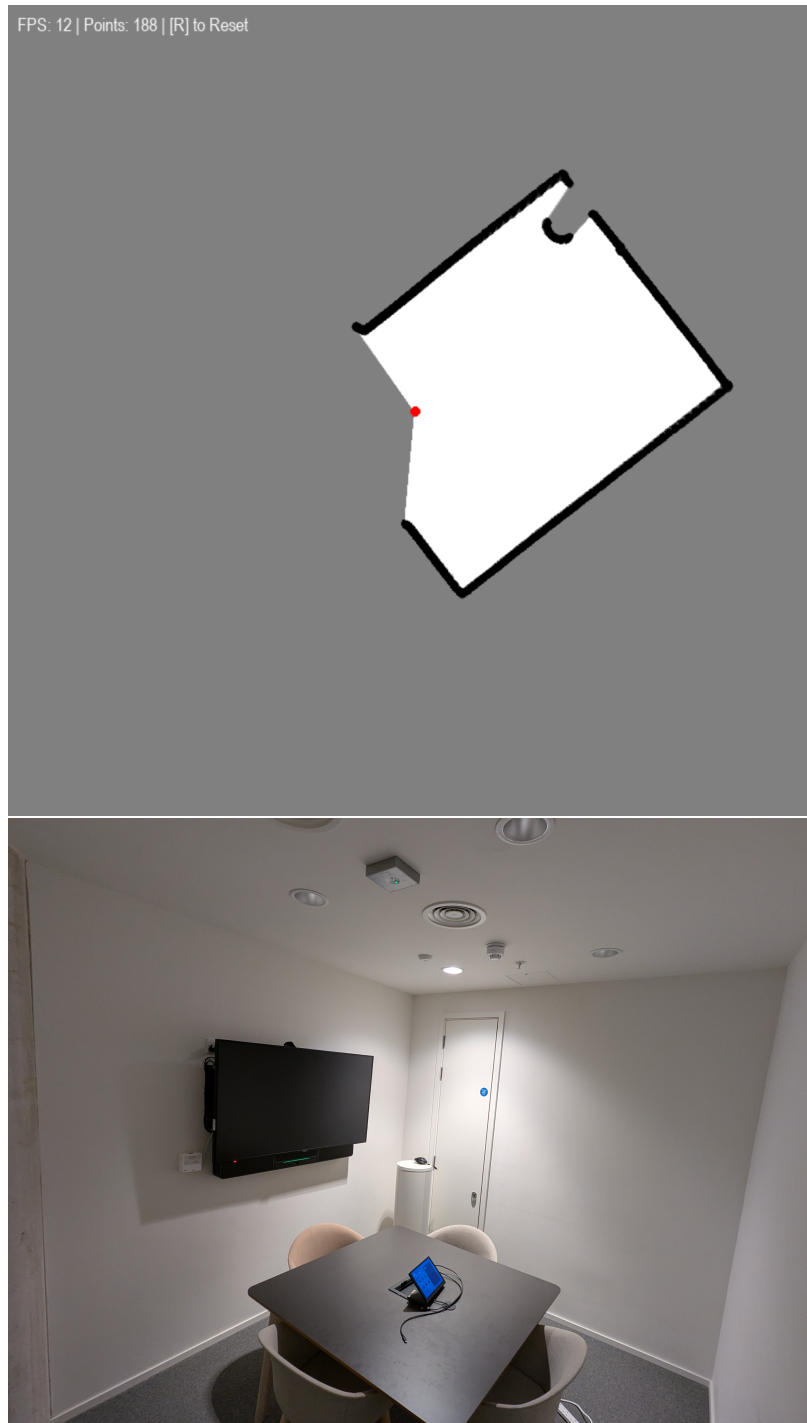
#### 3. Dynamic Updates: Ask a partner to walk in front of the sensor.

- Observation: You will see a black trail (representing the person) appear on the map.
- Action: Have the partner move away. Observe how the black trail slowly fades back to white/gray as new rays pass through that empty space, "erasing" the phantom obstacle.

Upon completing this activity, you will see a map that will look like the map shown in Fig. 3.

Question 1: Why is this grid superior to a Point Cloud for path planning? (Hint: Can a path planner calculate a route through "unknown" space?)

Question 2: For implementation, the range of 0.0 to 1.0 is mapped to integers from 0 to 255. In the Python file, locate the line marked with Activity 1. Change the `CONFIDENCE_FREE` variable. What do you observe?



**Figure 3: (top) An occupancy grid map generated from a set of scan, while the sensor was stationary, in a room. (bottom) The view of the room. The Lidar was about 1m from the floor.**

## Activity 2: Scan-to-Map Matching

### 5.1 Concept

Since we are using a handheld Lidar, we lack wheel encoders to measure distance ( $dx, dy$ ). We must infer motion using the Lidar data itself. We utilize Scan-to-Map Matching:

1. Take the current scan points.
2. Superimpose them onto the global map built in the previous frames.
3. "Wiggle" the scan coordinates  $(x, y, \theta)$  to find the alignment that maximizes the number of points landing on existing black pixels (Occupied Cells).

### 5.2 Instructions

1. Run the script `occupancy_grid_map_odometry.py`.
  2. Initialization: Hold the Lidar still for 5 seconds to let the initial map "burn in." The algorithm needs strong, high-confidence walls to lock onto.
  3. Rotation Test: Slowly rotate the Lidar  $45^\circ$  left and right.
    - Observation: The map should remain fixed on the screen (Allocentric view). The Red Dot (robot) should rotate in place.
  4. Translation Test: Slide the Lidar slowly across the table (approx. 30cm).
    - Observation: The Red Dot should move across the screen, leaving a blue trajectory line.
- Tips for Running the Algorithm:
    - Start Slowly: The scan matching assumes you don't teleport. Move the Lidar gently.
    - Environment: It works best in a room with corners and walls. It struggles in long, smooth hallways (nothing to "lock" onto) or open spaces.
    - Reset: If the map gets messy (which happens if you move too fast and the matching breaks), press `R` to restart the mapping. Real SLAM systems have "Loop Closure" to fix this, but this is a pure odometry script.
    - Avoid Roll and Pitch: The algorithm assumes that the sensor's motion is constrained to a 2D plane (3 Degrees of Freedom:  $x, y, \theta$ ). Consequently, only rotation about the vertical axis (Yaw) is valid. Ensure you hold the sensor level. Any tilting of the sensor (Roll or Pitch) will cause the laser to scan the floor or ceiling, introducing geometric errors into the map.
    - Discussion Question: How can we handle sensor motion that is not constrained to 2D? If roll and pitch are unavoidable (e.g., a handheld device or a drone), what additional hardware (e.g., IMU) or algorithmic steps would be needed to correct the laser scan data?

Upon completion of the activity, you should see the map and trajectory, as shown in Fig. 4.

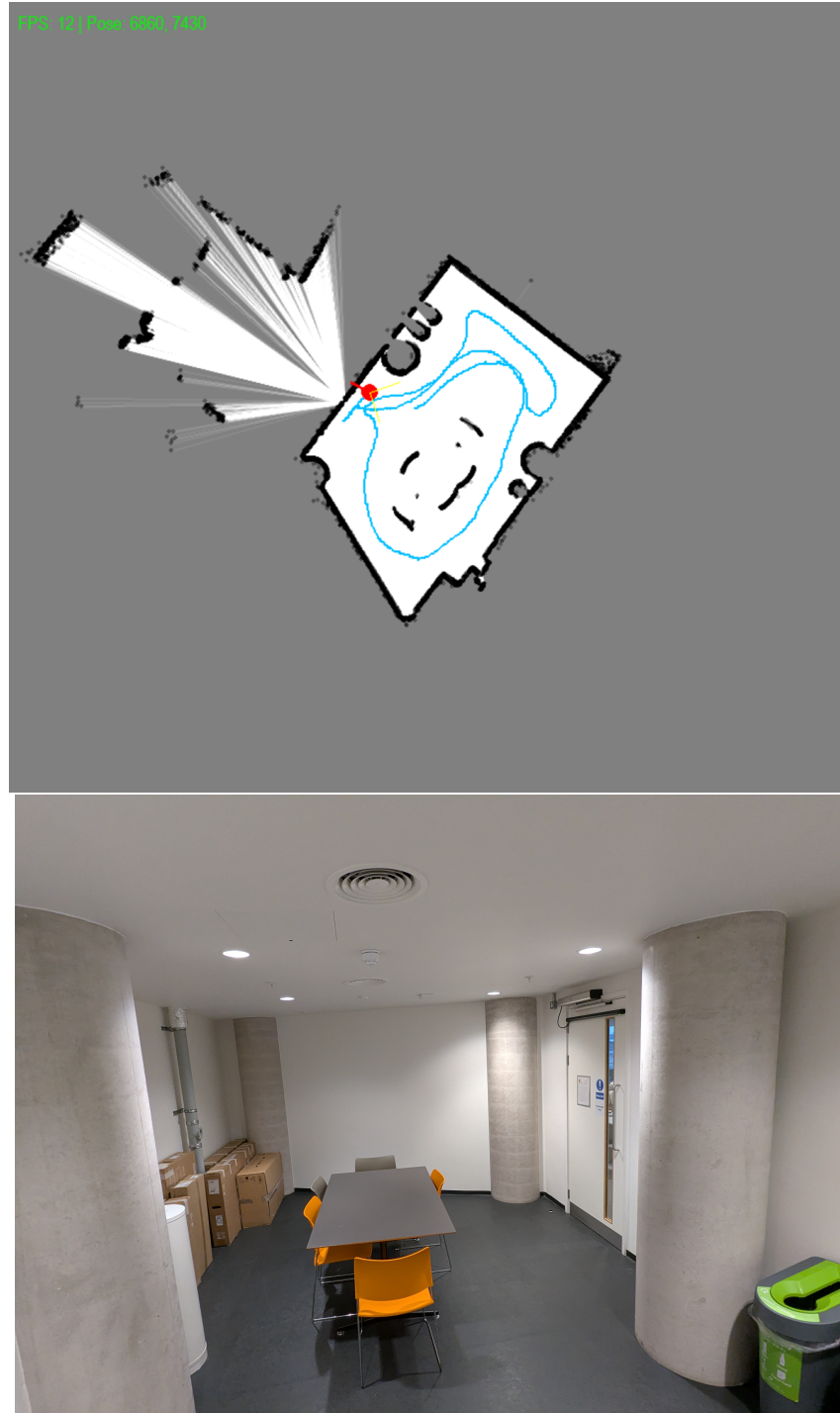


Figure 4: (top) Map (gray) and the trajectory (blue), generated by RPLIDAR A2M12 scan (red), moving in a room. The green circle shows the current location of the sensor. (bottom) The view of the room.

## Activity 3: Environmental Challenges

### The Corridor Problem (The Aperture Problem)

Lidar matching relies on distinct geometric features (corners, furniture) to lock its position along an axis.

1. **Experiment:** Take the sensor into a long, featureless hallway.
2. **Action:** Move the Lidar forward along the corridor axis.
3. **Observation:** You may notice the robot's position on the screen does not move forward, or drifts unpredictably.
4. **Analysis:** In a featureless tunnel, a scan taken at position  $X$  looks identical to a scan taken at  $X + 10cm$ . The error function is flat; the algorithm cannot find a unique minimum.

### Reflective Surfaces

1. **Experiment:** Point the Lidar at a glass window or computer monitor.
2. **Observation:** Note if points appear behind the glass (refraction/reflection) or if the beam is lost (specular reflection). Ghost walls can confuse the matching algorithm.

## Activity 4: Dynamic Limits

Our Python algorithm uses a discrete "Hill Climbing" search with a limited window (e.g.,  $\pm 30mm$  and  $\pm 0.5^\circ$  per frame).

1. **Experiment:** Rotate the Lidar very slowly; the map should hold. Now, snap your wrist quickly (high angular velocity).
2. **Observation:** The map will likely "break" or "double-expose."
3. **Analysis:** If the robot moves further than the search window in a single frame ( $1/60^{th}$  sec), the algorithm cannot find the true position and loses lock.

## Activity 5: Parameter Tuning (Recorded Data)

Adjust the constants in the Python script to observe their effects:

1. `CONFIDENCE_FREE`:
  - Set to Low (e.g., 1): Moving objects leave trails that persist for seconds.
  - Set to High (e.g., 50): The map becomes unstable; static walls might be erased by noise.
2. `iterations`:
  - Set to 1: High FPS, but lock breaks easily on rotation.
  - Set to 20: Robust tracking, but FPS drops significantly.

## A System Description

This appendix describes the odometry procedure.

### A.1 The Scan Matching Formulation

We aim to find the robot's pose  $\xi = (x, y, \theta)$  that maximizes the alignment between the current scan endpoints  $Z$  and the Map  $M$ .

We define a Score Function that represents the likelihood of the scan fitting the map:

$$\xi^* = \underset{\xi}{\operatorname{argmax}} \sum_{i=1}^N M(T(z_i, \xi)) \quad (1)$$

Where:

- $z_i$ : A single point from the laser scan in the robot's local frame  $(r, \phi)$ .
- $T(z_i, \xi)$ : The rigid body transformation converting the local point to Global Map coordinates:

$$\begin{bmatrix} x_{global} \\ y_{global} \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} z_x \\ z_y \end{bmatrix} + \begin{bmatrix} x \\ y \end{bmatrix} \quad (2)$$

- $M(\cdot)$ : The Occupancy Grid function. Returns the probability  $[0.0, 1.0]$  of a cell being occupied.

### A.2 Hill Climbing Optimization

Since calculating the gradient of a discrete grid map is complex, we use a discrete local search (Hill Climbing):

1. Initialization: Start with the pose from the previous frame:  $\xi_t \approx \xi_{t-1}$ .
2. Candidate Generation: Create a set of neighboring poses by perturbing the current estimate:

$$\text{Candidates} = \{(x \pm \delta, y, \theta), (x, y \pm \delta, \theta), (x, y, \theta \pm \epsilon)\}$$

3. Evaluate: Calculate the Score (sum of map probabilities) for all candidates.
4. Update: Move  $\xi$  to the candidate with the highest score.
5. Iterate: Repeat steps 2–4 for  $k$  iterations to converge on the local maximum within the timeframe of a single scan.